

Sovereign AI Workbench — Environment Setup Guide

Windows 11 · RTX 3050 6GB · Python 3.13 · For MRPL SIH Hackathon Team

Overview

This guide covers the one-time environment setup for the Sovereign On-Premise Agentic AI Workbench project, plus the routine to resume work in any new terminal session or after a reboot. Each team member should follow the First-Time Setup section once on their own laptop before Hour 0 of the hackathon clock.

Part A — First-Time Setup (do this once per laptop)

Step 1: Check your system

Confirm Python is installed and identify your GPU before installing anything.

```
python --version
(Windows) Check Device Manager > Display adapters for your GPU name
(If NVIDIA GPU) nvidia-smi — or full path: "C:\Program Files\NVIDIA
Corporation\NVSMI\nvidia-smi.exe"
```

Step 2: Install Ollama

Download and install the local model-serving runtime.

```
Download installer from https://ollama.com/download
Run the installer (installs silently, no wizard steps)
Open a NEW terminal window, then verify:
ollama --version
```

Step 3: Pull the two required models

These are the models your Model Router switches between. Combined download is ~8-9GB.

```
ollama pull gemma3:4b
ollama pull qwen2.5-coder:7b
Verify both are present:
ollama list
```

Step 4: Create project folder and virtual environment

Isolates all Python packages for this project.

```
mkdir mrpl-workbench
cd mrpl-workbench
python -m venv venv
Activate it:
.\venv\Scripts\Activate.ps1
If PowerShell blocks the script, run once:
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Step 5: Install Python packages

Installs Streamlit, ChromaDB, sentence-transformers, OCR bindings, docx writer, and torch (CPU build by default).

```
pip install streamlit ollama chromadb sentence-transformers pdfplumber PyPDF2 pytesseract pillow opencv-python python-docx --break-system-packages
```

Step 6: Install Tesseract OCR engine

pytesseract is only a Python wrapper — the actual OCR engine must be installed separately on Windows.

```
Download from https://github.com/UB-Mannheim/tesseract/wiki
Run the installer — keep default path: C:\Program Files\Tesseract-OCR
Leave "Additional script data" and "Additional language data" UNCHECKED (English-only is enough for this project)
Verify:
& "C:\Program Files\Tesseract-OCR\tesseract.exe" --version
```

Step 7: Sanity-check all imports

Confirms every package actually loads before you build anything on top of them.

```
python -c "import chromadb, sentence_transformers, docx, cv2, pytesseract, pdfplumber, streamlit, ollama; print('All core imports OK')"
```

Step 8 (optional): Enable GPU acceleration for embeddings

Not required — Ollama uses your GPU independently. Only do this if RAG ingestion feels slow later.

```
pip install torch==2.6.0+cu124 --index-url https://download.pytorch.org/whl/cu124
--force-reinstall --break-system-packages
Verify:
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

Step 9: End-to-end smoke test

Proves your two riskiest integration points — OCR and local LLM inference — work correctly together.

```
Create test_setup.py with:
import pytesseract, ollama
pytesseract.pytesseract.tesseract_cmd = r"C:\Program Files\Tesseract-OCR\tesseract.exe"
print(pytesseract.get_tesseract_version())
r = ollama.chat(model="gemma3:4b", messages=[{"role":"user","content":"Say OK if working."}])
print(r["message"]["content"])
Run it:
python test_setup.py
```

Step 10: Test the second model

Confirms qwen2.5-coder:7b also works — this is the model your router sends coding tasks to.

```
python -c "import ollama; r = ollama.chat(model='qwen2.5-coder:7b',
messages=[{'role':'user','content':'Write a one-line Python function that adds two numbers.'}]); print(r['message']['content'])"
```

Note: Step 8 may require finding the correct CUDA index tag for your Python version — check available builds with `pip index versions torch --index-url https://download.pytorch.org/whl/cu124` if the default fails. This step is a nice-to-have only; skip it and move on if it causes delays.

Part B — Resume Routine (every time you come back)

After the first-time setup is done, you never need to reinstall anything. Every new terminal session or reboot, just do this:

Step	Command	Why
1	<code>cd C:\mrpl-workbench</code>	Go to your project folder
2	<code>.\venv\Scripts\Activate.ps1</code>	Re-activate the virtual environment
3	<code>ollama list</code>	Confirm both models still show up

That's it — Ollama runs as a background service so your models are always loaded and ready; the venv keeps all your installed packages instantly available with no reinstalling.

Part C — Final Status Checklist

- ■ Python 3.13 + venv created
- ■ Ollama installed
- ■ gemma3:4b pulled and tested (responds correctly)
- ■ qwen2.5-coder:7b pulled and tested (responds correctly)
- ■ All Python packages installed and import cleanly
- ■ Tesseract OCR installed and verified from Python
- ■ GPU/CUDA confirmed active (RTX 3050 detected)
- ■ Model swap between the two models confirmed (VRAM constraint working as designed)

Part D — What Comes Next

Once every box above is checked, environment setup (Hour 0–2 in the project spec) is complete. The next phase is Hour 2–7: each team member builds and tests their own tool function in isolation (`ocr_extract`, `search_docs`, `run_code`, `write_docx`) against an agreed shared contract of `{"status", "data", "error"}` before anything gets wired together.